Project Title:

Prompt2Product – An Autonomous Agent for Natural Language to Executable Code

FYP Supervisor Name:

Sajid Ali

Reg ID & Faculty:

Reg ID: 2022669 Syeda Noor Fatima
Faculty: AI

1. Project Overview

Prompt2Product is an intelligent autonomous agent designed to convert plain English prompts into complete, executable software applications. The system accepts a high-level idea (e.g., “Build a task management web app”) and autonomously performs multi-step reasoning including requirement analysis, structured specification generation, full-stack code generation, dependency installation, execution, error detection, and iterative debugging.

Unlike single-response LLM systems, this project follows a multi-agent architecture where different models handle planning, specification design, coding, validation, and correction. The objective is to improve reliability, reduce hallucinations, enable tool integration, and ensure the generated application runs successfully without manual intervention.

The project contributes to Agentic AI research by exploring autonomous planning, tool usage, execution feedback loops, and fine-tuning strategies for improved code generation accuracy.

2. System Requirements

Hardware Requirements

- Minimum 16GB RAM system
- NVIDIA GPU with CUDA support
- At least 20GB available storage

GPU Requirement Justification

Currently, the allocated system contains a low-end GPU with 16GB RAM, which is sufficient for inference and small-scale experimentation. However, for fine-tuning large language models (LLMs), higher computational power and larger VRAM capacity are required.

Fine-tuning code generation models (7B+ parameters) requires high-end GPUs with larger VRAM (24GB–48GB or more) to handle:

- Large batch sizes
- Gradient updates
- Training checkpoints
- Efficient mixed-precision training

Therefore, access to a high-end GPU server is required to perform stable and efficient fine-tuning experiments as part of this FYP.

3. Software Requirements, Tools & Technologies

Programming & Backend

- Python 3.10+
- FastAPI
- Uvicorn

Frontend

- HTML, CSS, JavaScript
- Tailwind CSS

AI & LLM Tools

- Ollama (Local LLM serving)
- Qwen / Gemma (Code-capable models)
- LangGraph (Multi-agent orchestration)
- RAG pipeline for grounding

Development Tools

- VS Code
- Git & GitHub
- Docker (optional)

4. Framework Selection

LangGraph is selected to manage multi-step agent workflows including planning, code generation, execution, and correction loops. FastAPI is used for backend development due to its high performance and seamless Python integration. Ollama enables local GPU-based model execution.

This framework combination ensures modularity, scalability, and support for advanced research in autonomous AI systems.